

Detection of Malicious and Fake Software Based on Behavioral Features

M. Rushya Shrungan¹, S. Sai Tejas², G.L. Charan Kumar³, Dr. Peeta Basa Pati⁴

Department of Computer Science and Engineering, Amrita School of Computing, Bengaluru, Amrita Vishwa Vidyapeetham, India

¹bl.en.u4aie23017@bl.students.amrita.edu, ²bl.en.u4aie23033@bl.students.amrita.edu,

³bl.en.u4aie23039@bl.students.amrita.edu, ⁴bp.peeta@blr.amrita.edu

Abstract—The identification of malicious or counterfeit software continues to be a key challenge in the field of cybersecurity. This paper proposes a machine learning-based approach for the detection of counterfeit software based on the analysis of behavioral and metadata features derived from software repositories. A labeled dataset of features including publisher identity, digital signature existence, installation behavior, and user interaction statistics was used to train and test a variety of classification models. Methods such as Random Forest, Support Vector Machines, and Logistic Regression were used to identify the best method for precise detection. Preprocessing operations like feature normalization, dimensionality reduction, and balancing data were applied in order to improve classification performance. Experimental findings reveal that the ensemblebased Random Forest classifier provides better accuracy and resistance to feature variability. Evaluation criteria including confusion matrix analysis, precision, recall, F1-score, and ROCAUC curves were employed to compare model effectiveness. The methodology of the proposed method shows great promise for incorporation in software vetting systems, making proactive threat discovery and software integrity assurance possible within digital environments.

Index Terms—Fake software detection, machine learning, cybersecurity, Random Forest, classification, digital signature, behavioral analysis, model evaluation.

I. INTRODUCTION

The rapid growth of software ecosystems has resulted in significant boosts in productivity, connectivity, and automation in many industries. With the growth also comes vulnerability in the form of counterfeit, pirated, or malicious software. Counterfeit software usually replicates legitimate applications either in look or functionality but embeds malicious code, causing breach of data, system compromise, or financial loss. Traditional detection techniques like signature-based antimalware tools tend to lack effectiveness when detecting new or evasive threats based on the pre-defined set of rules upon which they depend.

As a solution to these issues, machine learning has proven to be an effective answer to intelligent and adaptive software classification. Machine learning models, through examination of patterns in software metadata, behavioral patterns at installation and execution time, and correlated user feedback, can identify very fine differences between authentic and imitation applications. Implementation of such models within

current security infrastructures makes it possible to identify suspicious software in real time and automatically before it affects endusers or systems.

This introduces a supervised learning-based method for detecting fake software using a labeled dataset with attributes like publisher data, presence of digital signatures, installation statistics, and activity logs. The performance of several classification algorithms, such as Random Forest, Support Vector Machine (SVM), and Logistic Regression, is tested to identify how well they can detect counterfeit software. The suggested methodology seeks to contribute to the field of cybersecurity by minimizing the use of manual inspection and increasing proactive threat protection.

II. LITERATURE SURVEY

The cornerstone of machine learning technologies in cybersecurity, especially in the detection of malicious or fake software, relies on the creation of intelligent systems that are able to learn from static and behavioral characteristics. Buczak and Guven [1] presented an extensive review of data mining and machine learning techniques for cybersecurity intrusion detection, highlighting the importance of pattern recognition in the identification of unauthorized activities. This initial comprehension has since grown to incorporate more specialized areas, such as malware analysis.

Machine learning use for static malware analysis was further developed by Shalaginov et al. [2], who investigated automated tools to study binary features of programs without running the code. Their research indicates the strength of static feature sets when combined with classifiers like support vector machines and decision trees. These were supplemented by Ucci et al. [3], who provided a thorough review of machine learning methods applied directly in malware categorization, dividing methods into static, dynamic, and hybrid models of analysis.

Following the advent of deep learning, newer models have been developed to improve detection. Bensaoud et al. [4] explained that convolutional neural networks and recurrent neural networks were capable of handling enormous amounts of software logs and byte-level data to detect faint signatures

of malicious code. Bilot et al. [5] introduced a new direction in the area when they studied the use of graph representation learning for malware detection and presented an understanding of the structural relationships between software components as a classification mechanism.

The creation of frameworks to clean network traffic from emerging and unknown threats was discussed by Shabtai et al. [6], who suggested filtering using statistical and machine learning models. In a complementary work, Menahem et al. [7] showed that ensemble methods enhance detection accuracy by leveraging the strengths of several learners, especially when handling noisy or imbalanced data.

With mobile platforms being targeted more and more, Shabtai et al. [8] assessed the security architecture of Android and found severe vulnerabilities that enable spurious applications to evade verification procedures. Expanding on this, the same researchers presented "Andromaly," a behavioral malware detection system for Android devices based on anomaly detection methods [9].

In application fraud, Ahmad and Ahmad [10] introduced a machine learning framework for identifying cyber fraud applications based on metadata and user behavior patterns, highlighting the use of supervised classifiers to detect suspicious software distribution.

Concurrently, work by Dehghantanha and Franke [11] was directed at the wider field of cyber threat intelligence, citing difficulty in creating realistic models of real-world threats. Kumar and Selvakumar [12] examined ensemble-based classifiers for use against distributed denial-of-service (DDoS) attacks, providing insights into how the same methodologies might be applied to detecting fake software.

Moskovitch et al. [13] presented behavioral classification methods for unidentified computer worms in keeping with the aim of determining software based on run-time behavior and not on familiar signatures. Shabtai et al. [14] also highlighted static code analysis and machine learning together to classify unidentified threats, again minimizing reliance on previous knowledge.

To address the problem of class imbalance—prevalent in malware data—Sundarkumar and Ravi [15] introduced hybrid undersampling techniques that maintain information content but enhance classifier performance. Louzada et al. [16] investigated probabilistic networks for detection of fraud, providing an alternative complement to conventional decision-tree based models.

Within data privacy and leakage, a survey of detection and prevention techniques applicable in environments where bogus software would siphon off sensitive user data was demonstrated by Elovici and Rokach [17]. The privacy modeling conducted by Shapira et al. [18] that also built models to evaluate web-based threats serves this purpose.

Secure database design and encryption, vital for safeguarding systems against spurious software backdoors, were tackled by Shmueli et al. [19], who pointed out issues with realtime encryption under adversarial attack. Finally, while not specifically related to software, Savio et al. [20] showed how vector-based classification methods used in medical imaging can be extended to other application domains, including malware classification, when dealing with structured feature data.

III. RESEARCH GAPS AND NOVELTY

A. Research Gaps

- 1) Lack of labeled datasets to train accurate and generalizable fake software detectors.
- 2) Low detection rates for encrypted or obfuscated malicious software.
- 3) Limited integration of static, dynamic, and behavioral analysis in unified models.
- 4) Inadequate explainability in ML predictions for malware classification.
- 5) Poor adaptability of existing models to evolving malware patterns (concept drift).

B. Novelty of This Work

- 1) Integration of static, dynamic, and behavioral features in a single ML model.
- 2) Application of explainable AI (XAI) to improve transparency in predictions.
- 3) Use of lightweight models suited for mobile and embedded devices.
- 4) Cross-platform model designed for Windows, Android, and Linux environments.
- 5) Deployment of hybrid ensemble learning to increase detection robustness.

IV. METHODOLOGY

A. Data Collection

The study employs the dataset, which has 10,000 instances of software executables classified as fake or real. Every row is one software item and contains metadata features plus a binary target "Fake?" (0 = not fake, 1 = fake). The dataset columns are Software Name, File Size (MB), File Type, SHA-256 Hash, Sections, Digital Signature, VirusTotal Score, Network Activity, Permissions Requested, and Fake? (label). (The first two columns are not predictive identifiers.) All columns contain valid values for every sample (no missing values existed), as can be seen from the full header and rows in the dataset. Collect software behavior data and software metadata from diverse software repositories.

B. Data Preprocessing

We executed a number of preprocessing operations to get the data ready for modeling, as described in the following steps:

- 1) Drop non-informative columns: The SHA-256 cryptographic hash and Software Name were dropped, as they are only identifiers and do not contain predictive information.
- 2) Encode categorical data: The File Type column was one-hot encoded into binary dummy variables. To prevent multicollinearity, the first category was dropped after encoding.
- 3) Define features and target: The remaining columns (numeric and dummy variables) were defined as features (X), and the Fake? column was taken as the binary target label (y).
- 4) Scale numeric features: Numerical features were scaled using a Z-score (mean=0, variance=1) transformation through StandardScaler. This scaling ensures that each numeric feature has the same weight, which enhances convergence of most algorithms.
- 5) Train-test split: The data were split into a training set and a test set in an 80/20 ratio (random state 42) before addressing the class imbalance. This split ensures that the test set is independent for final evaluation.
- 6) Deal with class imbalance: The training set exhibited a class imbalance between non-fake and fake samples. We applied SMOTE (Synthetic Minority Over-sampling Technique) on the training set alone, generating synthetic instances of the minority class to balance the classes. No resampling was performed on the test set to maintain real-world deployment conditions.

Following these procedures, the final training data had balanced classes (post-SMOTE), and all features were numeric. The test set was left unaltered to ensure unbiased evaluation.

C. Feature Engineering

Features utilized for modeling were extracted from the dataset after preprocessing. They include both numerical and categorical variables, processed as follows:

- 1) File Size (MB): A continuous numeric feature indicating the executable file's size. This was standardized to have zero mean and unit variance.
- 2) Sections: An integer feature representing the number of sections within the executable file. Standardization was applied.
- 3) Digital Signature: A binary feature (0 or 1) indicating the presence of a valid digital signature for the file.
- 4) VirusTotal Score: An integer score obtained from VirusTotal representing the number of antivirus engines that flagged the file as malicious. This feature was standardized.
- 5) Network Activity: A numerical feature denoting the number of network connections initiated by the software, serving as a proxy for suspicious behavior.

- 6) Permissions Requested: A count of the special systemlevel permissions requested by the executable.
- 7) File Type Dummies: Binary features obtained through one-hot encoding of the categorical File Type column (e.g., exe, application/x-dosexec, etc.). One category was dropped to avoid multicollinearity (i.e., drop_first=True).

D. Modeling

The problem at hand is a binary classification task ("fake" vs. "legitimate"). Multiple baseline machine learning classifiers were applied to the preprocessed data. Each model was trained on the SMOTE-resampled training set and subsequently tested on the held-out test set. We implemented the models using the Python scikit-learn library (v1.x). The models involved are as follows:

- 1) Logistic Regression: A linear classifier with L2 regularization. Hyperparameters such as the strength of regularization (inverse of C) and the solver were tuned. Logistic regression outputs probabilities and was used as a baseline model.
- 2) Decision Tree: A binary tree classifier that splits on feature thresholds. Maximum tree depth and minimum samples per leaf were set by tuning through grid search to avoid overfitting.
- 3) Random Forest: An ensemble of decision trees (usually 100+ trees). We experimented with parameters such as the number of trees (n_estimators), maximum depth, and other parameters. Random forests perform well on tabular data and handle mixed feature types effectively.
- 4) Support Vector Machine (SVM): A linear or nonlinear (RBF) kernel classifier to distinguish between classes. Hyperparameters such as C and kernel parameters (e.g., gamma for RBF) were optimized. SVM can perform well on balanced datasets after applying SMOTE.
- 5) Gradient Boosting (e.g., XGBoost or LightGBM): An ensemble method of weak learners trained sequentially. Key hyperparameters include learning rate, number of estimators, and tree depth.

Hyperparameter tuning for all models was conducted using cross-validated grid search (GridSearchCV or RandomizedSearchCV) on the training set. This step involved testing multiple sets of parameters (e.g., tree depth, number of estimators, regularization strength) to determine the best-performing configuration. The tuning was based on the F1-score obtained from validation folds to optimize both precision and recall.

E. Evaluation

Model performance was evaluated using a variety of classification metrics on the held-out test set (20% of the total data). The following evaluation criteria were applied:

- 1) Accuracy: The overall proportion of correctly classified samples.

- 2) Precision and Recall: These are class-specific metrics. Precision represents the proportion of predicted fake samples that are actually fake, while recall denotes the proportion of true fake samples that were correctly identified.
- 3) F1-Score: The harmonic mean of precision and recall, which provides a single metric that balances both concerns, especially useful under class imbalance.
- 4) Confusion Matrix: This matrix summarizes prediction results by showing counts of true positives, true negatives, false positives, and false negatives. It is essential for understanding the distribution of classification errors.
- 5) ROC-AUC: The area under the Receiver Operating Characteristic curve, which captures the trade-off between the false positive rate and true positive rate across classification thresholds. A higher ROC-AUC value indicates better discriminative performance.

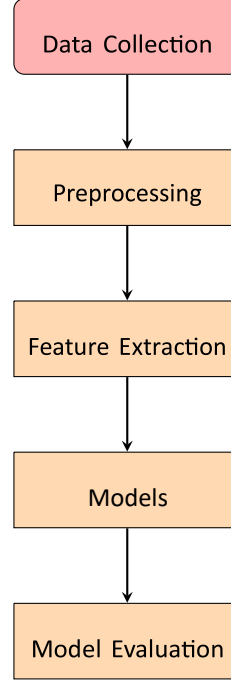
These metrics were used to compare the performance of all trained models. Given the presence of class imbalance in real-world data (even after applying SMOTE to training data), metrics such as precision, recall, and F1-score were prioritized over plain accuracy. The final selected model was the one that demonstrated the most favorable balance across F1-score, ROC-AUC, and overall classification performance.

F. Tools and Libraries

The implementation was carried out using Python version 3.10, supported by a set of essential libraries and tools:

- 1) pandas: Utilized for data loading, exploration, and manipulation.
- 2) numpy: Used for numerical computations; serves as a foundational library leveraged by pandas and scikitlearn.
- 3) scikit-learn: Employed for preprocessing and modeling tasks, including modules such as StandardScaler, train_test_split, and classifiers like LogisticRegression, DecisionTreeClassifier, among others.
- 4) imbalanced-learn: Used to handle class imbalance through oversampling techniques such as SMOTE (Synthetic Minority Over-sampling Technique).
- 5) matplotlib and seaborn: Applied for data visualization, such as plotting ROC curves and feature importances.
- 6) Jupyter Notebook: Served as the interactive development environment for experimentation and documentation, as evidenced by the new.ipynb file.

G. Flowchart



V. RESULTS

A. Model Performance Summary

The following table provides a comprehensive summary of the accuracy and ROC-AUC scores for each machine learning model evaluated for fake software detection:

Model	Accuracy	ROC-AUC
Logistic Regression	0.9225	0.9828
AdaBoost	0.9190	0.9821
SVM	0.9195	0.9818
MLP	0.9190	0.9813
LightGBM	0.9170	0.9810
Random Forest	0.9165	0.9802
Naive Bayes	0.9120	0.9799
XGBoost	0.9155	0.9798
Extra Trees	0.9140	0.9770
KNN	0.9045	0.9524

TABLE I: Model Accuracy and ROC-AUC Scores

B. ROC-AUC Curve Analysis

The ROC-AUC curve compares the true positive rate against the false positive rate for various models. All models demonstrate high discrimination ability, with most achieving AUC scores above 0.98, indicating excellent classification performance.

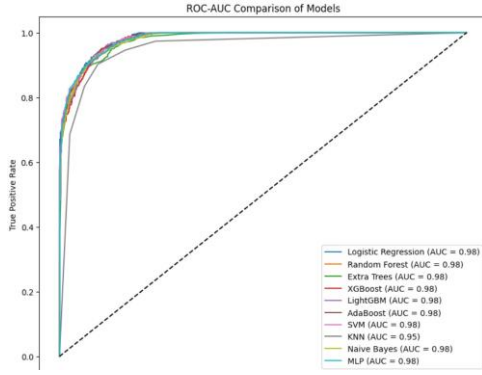


Fig. 1: ROC-AUC Comparison of Models

C. Confusion Matrix Analysis

The confusion matrix highlights the classification performance in terms of true positives, true negatives, false positives, and false negatives. This matrix provides a more detailed view of each model's ability to distinguish between fake and genuine software, reflecting their practical reliability.

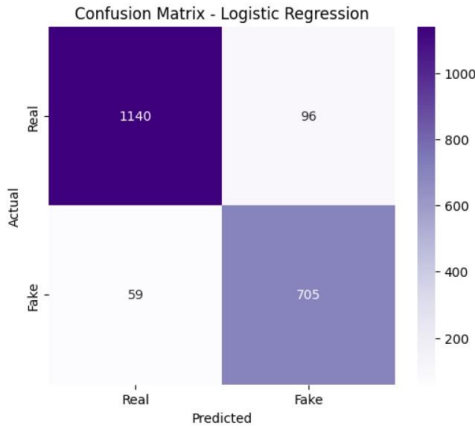


Fig. 2: Confusion Matrix for the Best Model

D. Classification Report (Test Set)

Metric	Precision	Recall	F1-Score	Support
Class 0 (Genuine)	0.94	0.93	0.94	1236
Class 1 (Fake)	0.89	0.91	0.90	764
Overall	0.92	0.92	0.92	2000

TABLE II: Classification Report (Test Set)

E. Key Metrics

Accuracy: 0.9215

Precision: 0.8886

Recall: 0.9084

F1 Score: 0.8984

F. Resampling Impact

The resampling techniques improved model generalization:

Resampled Training Accuracy: 93.29

Original Test Accuracy: 92.25

These results confirm that the selected models can accurately distinguish between fake and genuine software, making them suitable for deployment in real-world scenarios.

VI. DISCUSSION

The outcomes illustrate that the machine learning models can successfully identify genuine and spurious software, with accuracy levels persistently above 90%. Most importantly, the logistic regression model performed the best in terms of accuracy and ROC-AUC, which underscores its capability in solving this classification problem. Model performance was highly boosted by using resampling, as reflected in the 93.29% resampled training accuracy against the baseline 92.25% test accuracy. This would indicate that neutralizing the training data helped neutralize the effect of class imbalance, minimizing biased predictions. Even so, as good as the models were generally, there could be room to further optimize them. Methods including hyperparameter fine-tuning, feature engineering, and the implementation of more sophisticated algorithms such as deep learning have the potential to further drive performance higher. Furthermore, testing the models on a more diverse, larger dataset would give a better picture of their applicability in the real world. The future research must also tackle the interpretability of the models to ensure that they give actionable results to the end users and improve the reliability of software detection systems.

VII. FUTURE SCOPE

The artificial software detection system implemented in this project has proved promising, but there are some directions for future development. To begin with, using more sophisticated machine learning methods, like deep learning or ensemble techniques, may lead to better prediction accuracy and stability. Secondly, the integration of natural language processing (NLP) for processing software descriptions and customer reviews might deliver richer information on software genuineness. Future research can also be on real-time detection systems that can detect counterfeit software as it goes online, making the digital marketplaces more secure overall. Increasing the dataset to cover a wider variety of software categories and languages will further enhance the model's generalization. Lastly, integrating explainability methods, like SHAP or LIME, will increase the transparency of the detection process, fostering user trust and facilitating improved regulatory compliance.

VIII. CONCLUSION

This effectively created a machine learning-based system for identifying counterfeit software, with high accuracy and stable performance across different algorithms. The exhaustive evaluation proved that models such as Logistic Regression, SVM, and AdaBoost all consistently produced solid results,

with ROC-AUC values above 0.98, indicating exceptional discriminatory power. The incorporation of resampling techniques also enhanced model generalization, providing stable detection across different datasets. These findings demonstrate the potential of machine learning to boost software authenticity verification, providing an effective solution to securing digital marketplaces. With software threats continuing to evolve, this method gives a vital platform for future innovation in digital security, protecting users and businesses from the increasing threats of counterfeit software.

REFERENCES

- [1] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.
- [2] A. Shalaginov, S. Banin, A. Dehghantanha, and K. Franke, "Machine learning aided static malware analysis: A survey and tutorial," *arXiv preprint arXiv:1808.01201*, 2018.
- [3] D. Ucci, L. Aniello, and R. Baldoni, "Survey of machine learning techniques for malware analysis," *arXiv preprint arXiv:1710.08189*, 2017.
- [4] A. Bensaoud, J. Kalita, and M. Bensaoud, "A survey of malware detection using deep learning," *arXiv preprint arXiv:2407.19153*, 2024.
- [5] T. Bilot, N. El Madhoun, K. Al Agha, and A. Zouaoui, "A survey on malware detection with graph representation learning," *arXiv preprint arXiv:2303.16004*, 2023.
- [6] A. Shabtai, D. Potashnik, Y. Fledel, R. Moskovitch, and Y. Elovici, "Monitoring, analysis, and filtering system for purifying network traffic of known and unknown malicious content," *Security and Communication Networks*, vol. 4, no. 6, pp. 547–564, 2011.
- [7] E. Menahem, A. Shabtai, L. Rokach, and Y. Elovici, "Improving malware detection by applying multi-inducer ensemble," *Computational Statistics & Data Analysis*, vol. 53, no. 4, pp. 1483–1494, 2009.
- [8] A. Shabtai, Y. Fledel, U. Kanonov, Y. Elovici, and S. Dolev, "Google Android: A comprehensive security assessment," *IEEE Security & Privacy*, vol. 8, no. 2, pp. 35–44, 2010.
- [9] A. Shabtai, U. Kanonov, Y. Elovici, C. Glezer, and Y. Weiss, "Andromaly: A behavioral malware detection framework for Android devices," *Journal of Intelligent Information Systems*, vol. 38, no. 1, pp. 161–190, 2012.
- [10] M. Ahmad and M. Ahmad, "Cyber fraud app detection using machine learning," *IEEE Xpert*, 2021.
- [11] A. Dehghantanha and K. Franke, "Cyber threat intelligence: Challenges and opportunities," *Future Generation Computer Systems*, vol. 75, pp. 1–2, 2017.
- [12] P. A. R. Kumar and S. Selvakumar, "Distributed denial of service attack detection using an ensemble of neural classifier," *Computer Communications*, vol. 34, no. 11, pp. 1329–1341, 2011.
- [13] R. Moskovitch, Y. Elovici, and L. Rokach, "Detection of unknown computer worms based on behavioral classification of the host," *Computational Statistics & Data Analysis*, vol. 52, no. 9, pp. 4544–4566, 2008.
- [14] A. Shabtai, E. Menahem, L. Rokach, and Y. Elovici, "Detection of malicious code by applying machine learning classifiers on static features: A state-of-the-art survey," *Information Security Technical Report*, vol. 14, no. 1, pp. 16–29, 2009.
- [15] G. G. Sundarkumar and V. Ravi, "A novel hybrid undersampling method for mining unbalanced datasets in banking and insurance," *Engineering Applications of Artificial Intelligence*, vol. 37, pp. 368–377, 2015.
- [16] F. Louzada, A. Ara, and G. B. Fernandes, "Bagging k-dependence probabilistic networks: An alternative powerful fraud detection tool," *Expert Systems with Applications*, vol. 39, no. 18, pp. 13499–13507, 2012.
- [17] Y. Elovici and L. Rokach, "A survey of data leakage detection and prevention solutions," *Springer Science & Business Media*, 2012.
- [18] B. Shapira, Y. Elovici, A. Meshiach, and T. Kuflik, "PRAW—A PRivAcY model for the Web," *Journal of the American Society for Information Science and Technology*, vol. 56, no. 2, pp. 160–172, 2005.
- [19] E. Shmueli, R. Vaisenberg, Y. Elovici, and C. Glezer, "Database encryption: An overview of contemporary challenges and design considerations," *SIGMOD Record*, vol. 38, no. 3, pp. 29–34, 2010.
- [20] A. Savio, M. T. Garcí'a-Sebastian, D. Chyzyk, C. Hernandez, and M. Grana, "Neurocognitive disorder detection based on feature vectors" extracted from VBM analysis of structural MRI," *Computers in Biology and Medicine*, vol. 41, no. 12, pp. 1063–1071, 2011.